



Agentic Platform

PLATAFORMA AGÉNTICA MULTI-TENANT

MANUAL 01 · MANUAL DE USUARIO

Proyectos

Dirigido a: **Administradores de tenant (tenant_admin) y operadores que gestionan los proyectos de su organización en el panel de administración.**

Este manual explica cómo trabajar con **Proyectos** en el panel de administración de la plataforma agéntica. Un proyecto es la unidad de trabajo donde viven los planes, tareas, conversaciones con agentes, comandos autorizados, servidores MCP, bases de conocimiento, memorias del equipo, webhooks entrantes y la configuración de git y de validación humana.

Aprenderás a localizar y consultar la lista de proyectos de tu tenant, a crear un proyecto nuevo paso a paso mediante el asistente (partiendo de una plantilla o en blanco), y recorrerás un proyecto real de ejemplo («**Hello World PHP**») sección por sección: el hub, la edición, el app-preview de validación, el chat con los agentes, los planes, las tareas, el conocimiento RAG, los comandos y runtimes, la caché de dependencias, los servidores MCP, los webhooks, la memoria y el diagnóstico de herramientas.

Generado · 2026-07-18

Idioma · Español

Versión · Producción

Confidencial · Comité de Dirección

Contenido

- 01** Lista de proyectos
- 02** Asistente de creación — Paso 1: elegir plantilla o empezar en blanco
- 03** Asistente de creación — Paso 2: personalizar y crear
- 04** El hub del proyecto «Hello World PHP»
- 05** Editar el proyecto: campos básicos
- 06** App-preview de validación humana (imagen y puerto)
- 07** Chat con los agentes del proyecto
- 08** Planes del proyecto
- 09** Tareas del proyecto
- 10** Bases de conocimiento del proyecto (RAG)
- 11** Comandos autorizados y runtime por defecto
- 12** Caché de dependencias por runtime
- 13** Servidores MCP del proyecto
- 14** Webhooks entrantes del proyecto
- 15** Memoria del proyecto
- 16** Diagnóstico de herramientas por agente

1 Lista de proyectos

Esta es la pantalla principal de **Proyectos** (grupo *Recursos* de la navegación lateral). Muestra los proyectos activos de tu tenant en una cuadrícula de tarjetas; las plantillas no aparecen aquí — se eligen al crear un proyecto nuevo.

Cada tarjeta muestra el **nombre** del proyecto, una **insignia de estado** (**active** en verde, **paused** en ámbar, **archived** en gris) y la **descripción**. Haz clic en cualquier tarjeta para abrir el **hub** de ese proyecto, la ficha desde la que se accede a todas sus secciones.

En la esquina superior derecha, los usuarios con rol **administrador de tenant** o superior ven el botón «**Crear proyecto**», que lanza el asistente de creación en dos pasos.

Buena práctica: usa el estado para mantener la lista útil — pausa los proyectos aparcados y archiva los terminados; así la operativa diaria se concentra en lo activo.

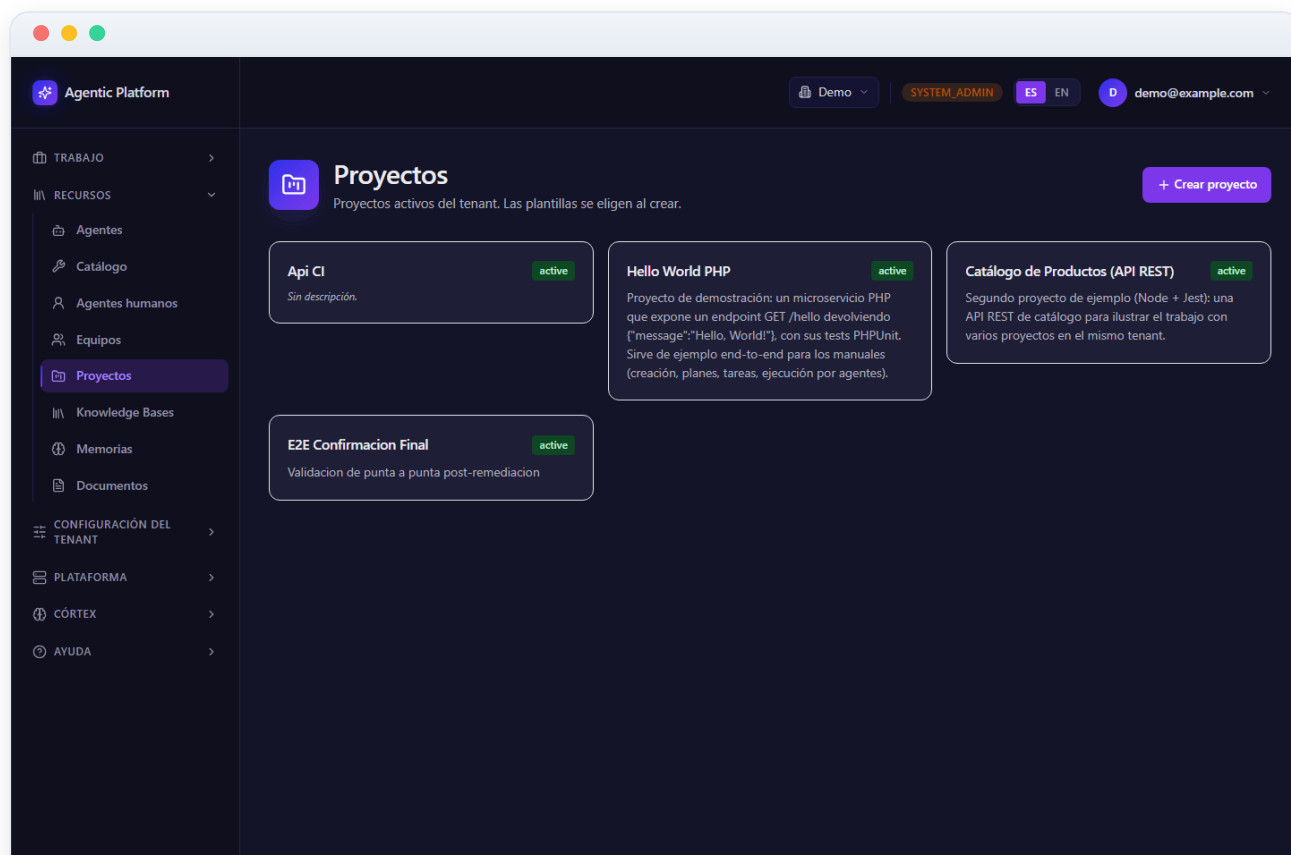


Figura 01.1 — Lista de proyectos

2 Asistente de creación — Paso 1: elegir plantilla o empezar en blanco

Al pulsar «Crear proyecto» entras en el **asistente de dos pasos**. El paso 1 decide el punto de partida:

- **«Proyecto en blanco»** (tarjeta superior): empieza sin plantilla. No se concede ninguna base de conocimiento por defecto y el equipo lo eliges tú en el paso 2. Pulsa **«Empezar en blanco»** para continuar.
- **Plantillas** (cuadrícula inferior): proyectos preconfigurados que traen equipo de agentes, política de validación humana, configuración de repositorio y bases de conocimiento por defecto. Cada tarjeta muestra el nombre, el **equipo** asociado (insignia azul) y la descripción. Pulsa **«Usar plantilla»** para seleccionarla — el nombre y la descripción del proyecto se precargan a partir de la plantilla.

Cuándo usar cada opción: las plantillas son la vía rápida para stacks conocidos (traen el equipo y las convenciones listas); el proyecto en blanco es para casos a medida donde prefieres montar cada pieza tú mismo.

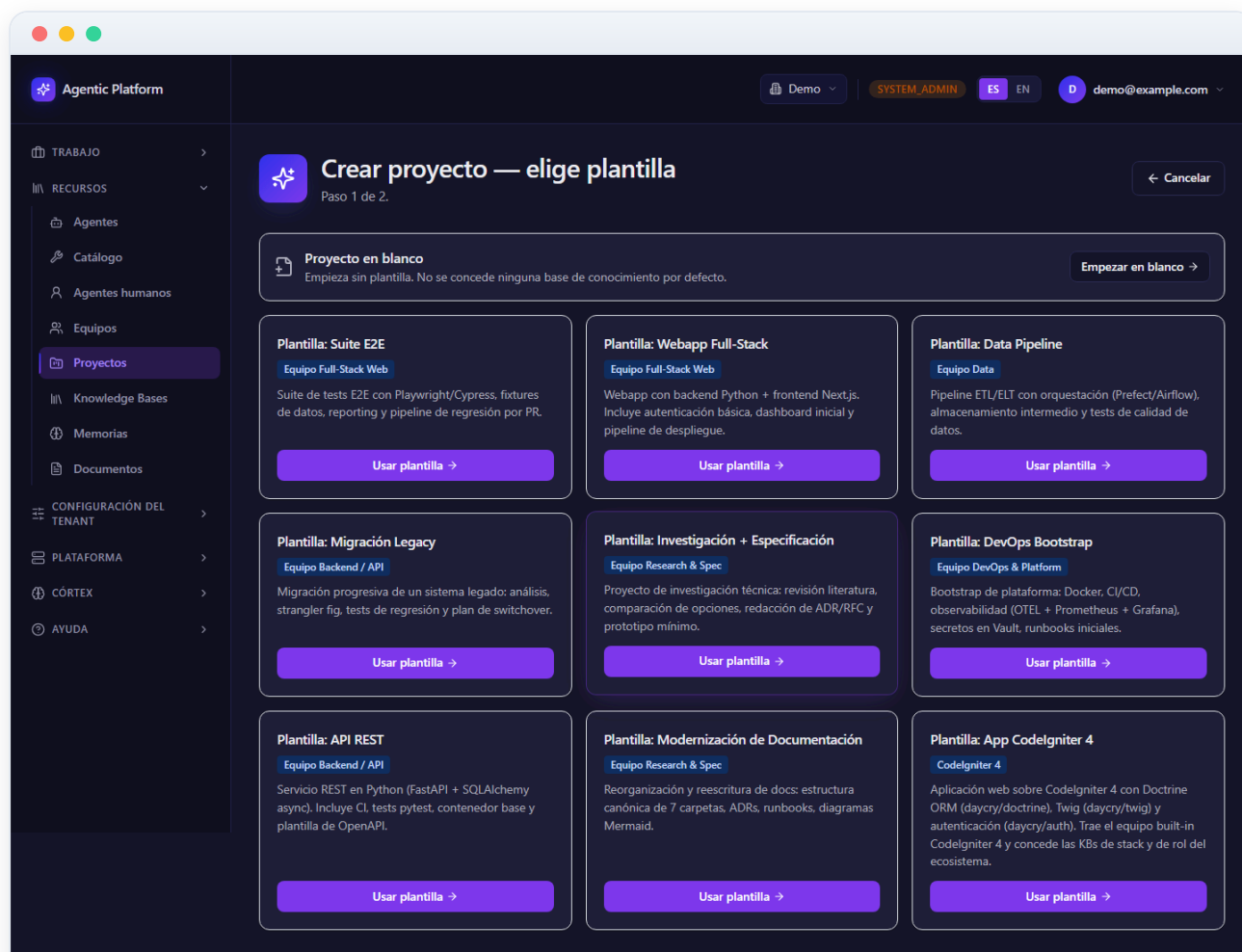


Figura 01.2 — Asistente de creación — Paso 1: elegir plantilla o empezar en blanco

3 Asistente de creación — Paso 2: personalizar y crear

El paso 2 ajusta los detalles antes de crear. El panel «**Detalles del proyecto**» contiene:

- **Nombre** (obligatorio) y **Descripción** (con soporte Markdown y previsualización).
- **Runtime por defecto**: un desplegable con el catálogo real de runtime templates del stack (para este ejemplo elegiríamos `PHP` · `PHPUnit`). Puede dejarse vacío — cada herramienta usará entonces su runtime por defecto.
- Si empezaste **en blanco**: un selector de **Equipo** (el equipo gobierna qué agentes ejecutan las tareas y la política de memoria; puedes asignarlo después desde la ficha).
- Si elegiste **plantilla**: la casilla «**Conceder las bases de conocimiento de la plantilla**» (activada por defecto; si la desmarcas, el proyecto adopta la plantilla pero sin KBs).

El panel «**Preview**» de la derecha resume lo que llevará el proyecto: la plantilla elegida (o «en blanco»), el **equipo**, la casilla «**Personalizar el equipo para este proyecto**» (crea una copia editable del equipo en lugar de referenciar el compartido), la **política de validación humana** heredada y la configuración de **repositorio**.

El botón «**Crear proyecto**» se habilita cuando hay nombre; al crearlo, vuelves a la lista con el proyecto nuevo disponible. El botón «← **Cambiar plantilla / Volver**» regresa al paso 1 sin perder lo tecleado.

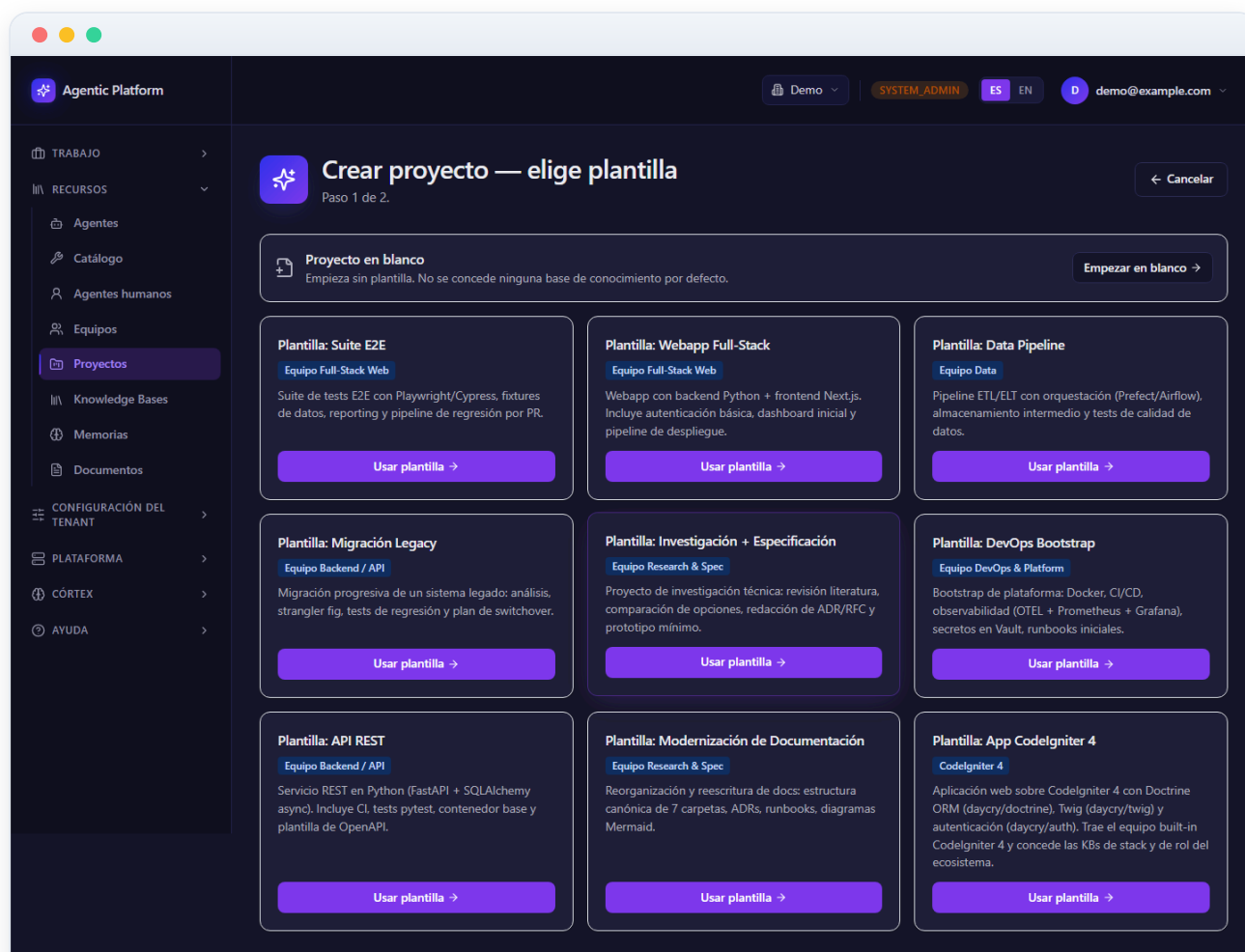


Figura 01.3 — Asistente de creación — Paso 2: personalizar y crear

4 El hub del proyecto «Hello World PHP»

Al abrir un proyecto llegas a su **hub**: la ficha central desde la que se accede a todo lo demás. De arriba abajo encontrarás:

- **Cabecera**: el nombre del proyecto, su descripción y los botones «**Editar**» (campos básicos) y «**Borrar**» (con confirmación escribiendo el nombre; borra planes, tareas y conversaciones — los repos git en disco no se tocan).
- **Fila de estado**: la insignia de estado (`active` , `paused` , `archived`), si es plantilla y el equipo asignado.
- **Hub de capacidad**: un resumen de qué *sabe* el proyecto (bases de conocimiento concedidas) y qué *recuerda* (ámbitos de memoria), con el estado de configuración de cada sección.
- **Modelo del proyecto y Modelo del chat**: el proveedor y modelo LLM por defecto para la ejecución de agentes y para el chat. Vacío = heredar del nivel superior (equipo → plataforma), siguiendo la herencia de modelos de la plataforma.
- **Configuración Git**: remoto, rama por defecto, autenticación (PAT o clave SSH) y las políticas del flujo git del plan.
- **App-preview de validación humana**: la imagen y el puerto con los que se levanta la app del proyecto cuando un plan llega a validación humana (lo vemos en detalle más adelante).
- **Secciones**: la cuadrícula de tarjetas de acceso a cada sub-página (Chat, Planes, Tasks, Knowledge Bases, Memoria, MCP servers, Tools por agente, Comandos & runtime, Caché de dependencias y Webhooks entrantes).

Este proyecto de ejemplo expone un microservicio PHP con un endpoint `GET /hello` ; lo usaremos como hilo conductor para recorrer planes, tareas, conocimiento y ejecución.

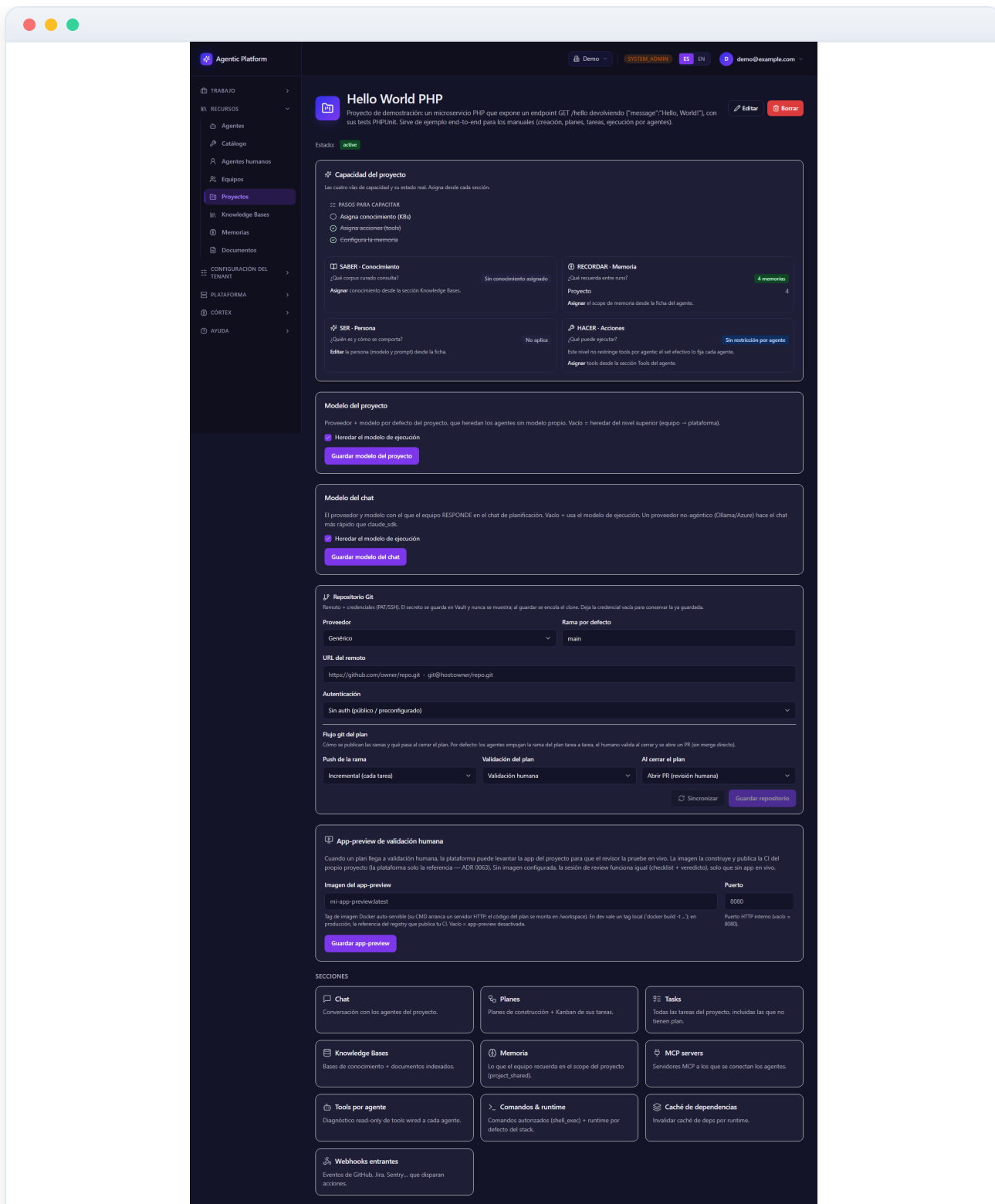


Figura 01.4 — El hub del proyecto «Hello World PHP»

5 Editar el proyecto: campos básicos

El botón «**Editar**» de la cabecera abre el diálogo de edición de los **campos básicos** del proyecto:

- **Nombre** (obligatorio) y **Descripción** — el editor de descripción soporta Markdown con previsualización.
- **Estado**: *Activo* (operativa normal), *Pausado* (trabajo suspendido temporalmente) o *Archivado* (cerrado; deja de aparecer en las vistas operativas).
- **Equipo**: el equipo de agentes del proyecto. Governa qué agentes ejecutan sus tareas y la política de memoria. Puedes dejarlo «Sin equipo» y asignarlo más tarde.

La configuración avanzada (servidores MCP, bases de conocimiento, comandos...) no se edita aquí: cada área tiene su propia sub-sección en el hub. Pula «**Guardar**» para aplicar los cambios o «**Cancelar**» para descartarlos.

Aviso: el botón «Borrar» de la cabecera es irreversible y exige teclear el nombre exacto del proyecto como confirmación — un seguro contra borrados accidentales.

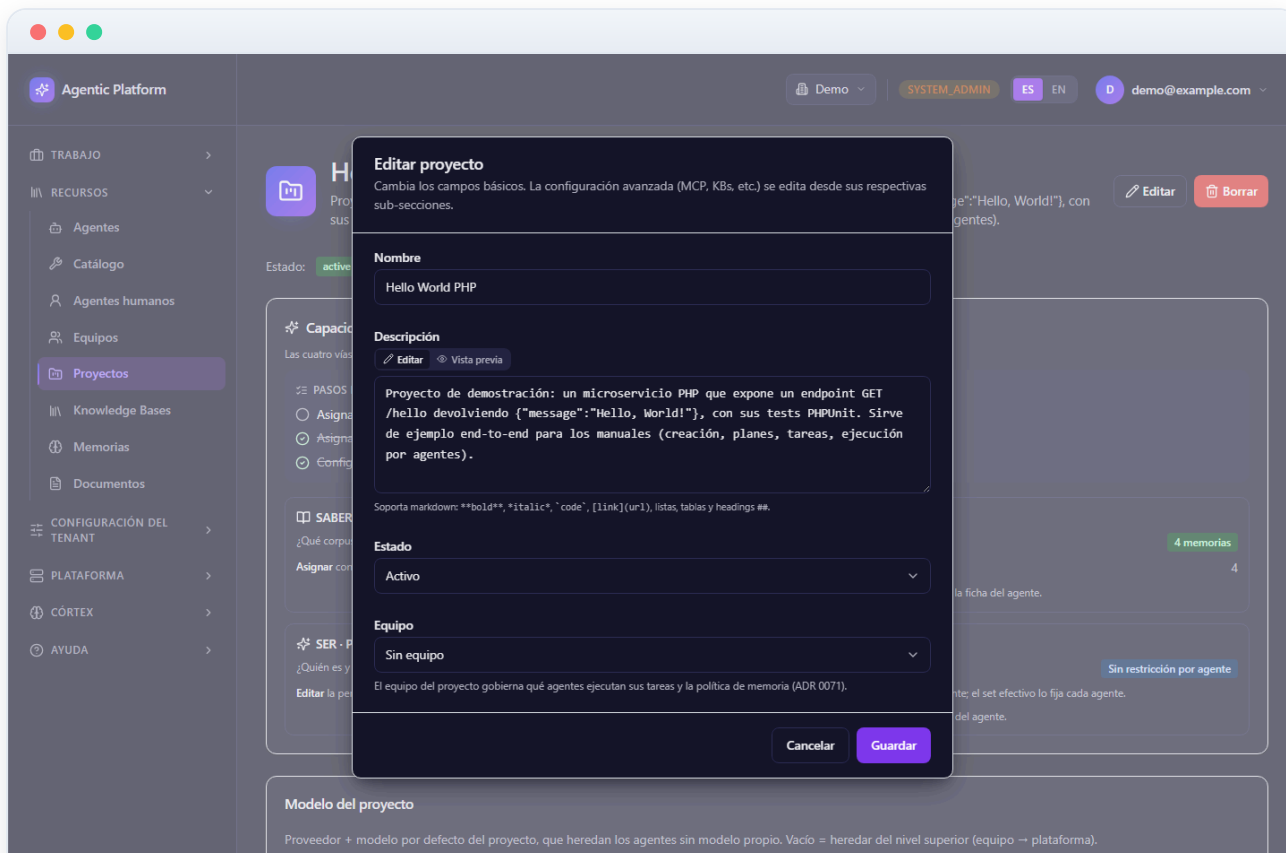


Figura 01.5 — Editar el proyecto: campos básicos

6 App-preview de validación humana (imagen y puerto)

La tarjeta «**App-preview de validación humana**» configura la demo en vivo que ve el revisor cuando un plan llega a validación humana: la plataforma levanta la app del proyecto en un contenedor de revisión para que se pueda **probar de verdad** antes de aprobar o rechazar el plan.

- **Imagen del app-preview:** el tag de una imagen Docker auto-servible (su `CMD` arranca un servidor HTTP; el código del plan se monta en `/workspace`). La imagen la construye y publica la CI del propio proyecto — la plataforma solo la referencia. En desarrollo vale un tag local; en producción, la referencia del registry.
- **Puerto:** el puerto HTTP interno en el que escucha la app (vacío = 8080).

Si dejas la imagen vacía, el **app-preview queda desactivado**: la sesión de validación humana funciona igualmente (checklist + veredicto), solo que sin app en vivo que probar.

Buena práctica: configura la imagen desde el primer plan; probar la app real reduce drásticamente los rechazos tardíos.

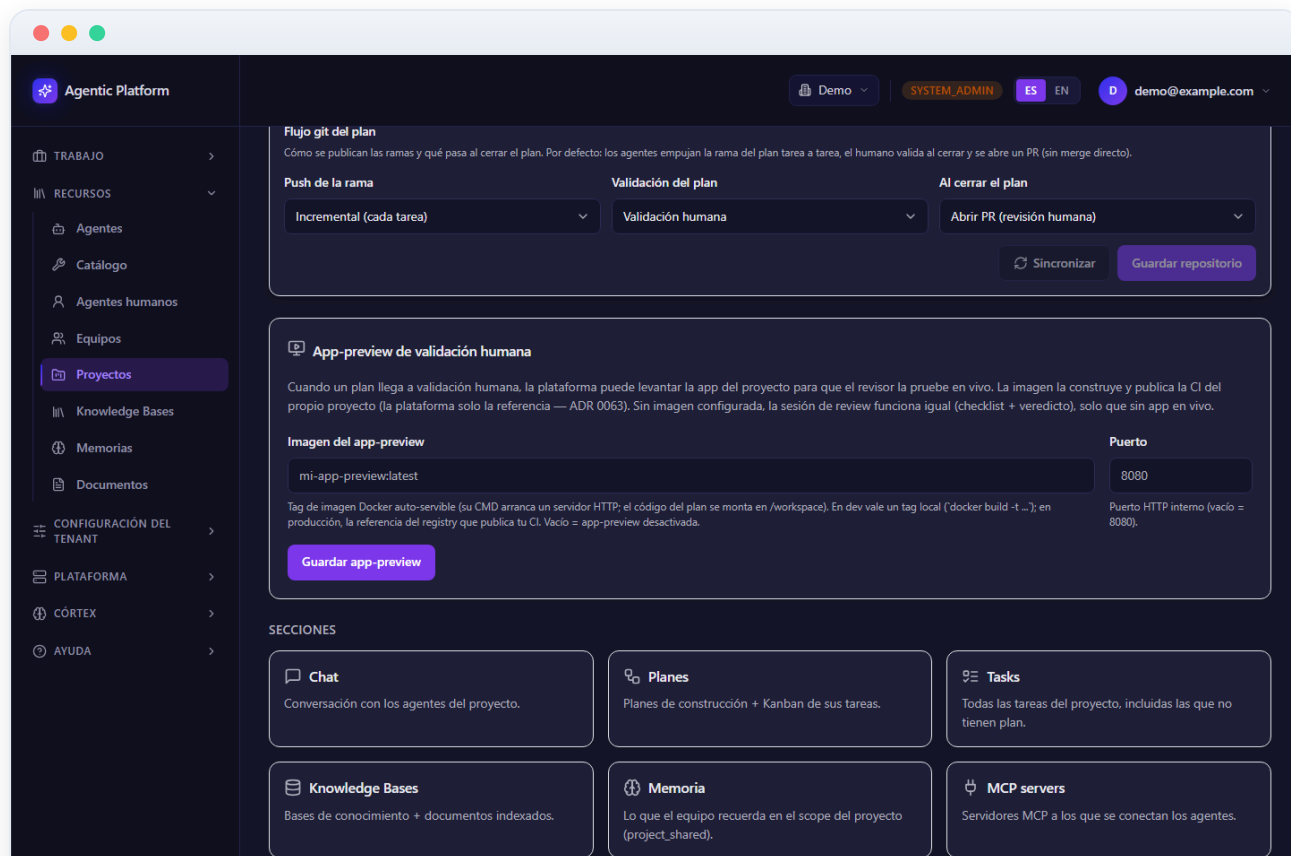


Figura 01.6 — App-preview de validación humana (imagen y puerto)

7 Chat con los agentes del proyecto

La sección **Chat** es la conversación con el equipo de agentes del proyecto y el punto de partida natural de todo el trabajo. Desde aquí pides lo que necesitas en lenguaje natural («crea el endpoint y su test») y el sistema lo convierte en **planes** y **tareas** ejecutables.

- **Modos de conversación:** la cabecera tiene un selector de modo persistente — *Planning* (diseñar un plan de construcción), *Discusión* (consultar sin generar trabajo), *Ejecución* y *Custom*. En modo Planning, la conversación termina proponiendo un plan con fases, tareas y estimaciones que puedes insertar en el proyecto.
- **@-menciones:** puedes dirigirte a un especialista concreto del equipo (project_manager, architect, backend_dev, frontend_dev, qa, reviewer, devops, security...) escribiendo @ en el compositor.
- **Historial:** el proyecto conserva sus conversaciones; puedes retomar una anterior o empezar una nueva.

Cada mensaje queda asociado al proyecto y a su contexto: las bases de conocimiento concedidas, la memoria del equipo y las herramientas autorizadas. Los agentes responden con ese contexto, no en el vacío.

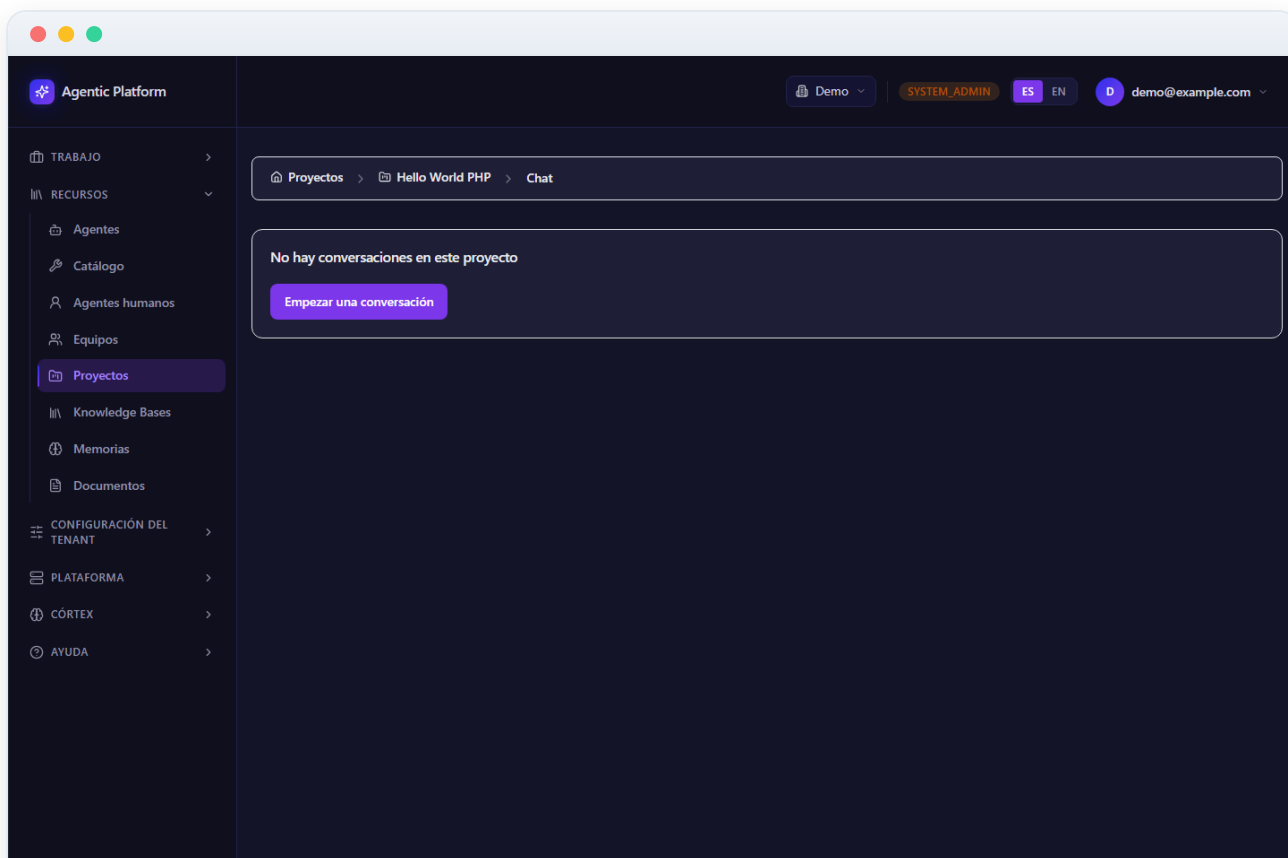


Figura 01.7 — Chat con los agentes del proyecto

8 Planes del proyecto

La sección **Planes** lista los planes de construcción del proyecto. Cada plan es la **unidad de cambio**: agrupa un conjunto ordenado de tareas con dependencias, se materializa como una rama git `plan/<id>-<slug>` y, al completarse, genera un PR automático. Verás el plan «MVP — API Hello World en PHP» creado para este ejemplo.

- **Filtros por estado**: la fila de chips filtra por el ciclo de vida completo — Borrador, Pendiente de aprobación, Aprobado, En progreso, Bloqueado, Pendiente validación humana, Completado, Rechazado, Cancelado y Archivado — con el contador de planes en cada estado.
- **Dos vistas**: lista (una tarjeta por plan con título, descripción e insignia de estado) o kanban por estado, conmutables desde la cabecera.
- **«Generar desde chat»**: acceso directo al chat en modo Planning para crear un plan nuevo conversando con el equipo.

Al hacer clic en un plan accedes a su **vista de detalle** (resumen, fases, tareas, DAG, Gantt, costes, sincronización al Kanban...), que se documenta a fondo en el manual de **Planes y Kanban**.

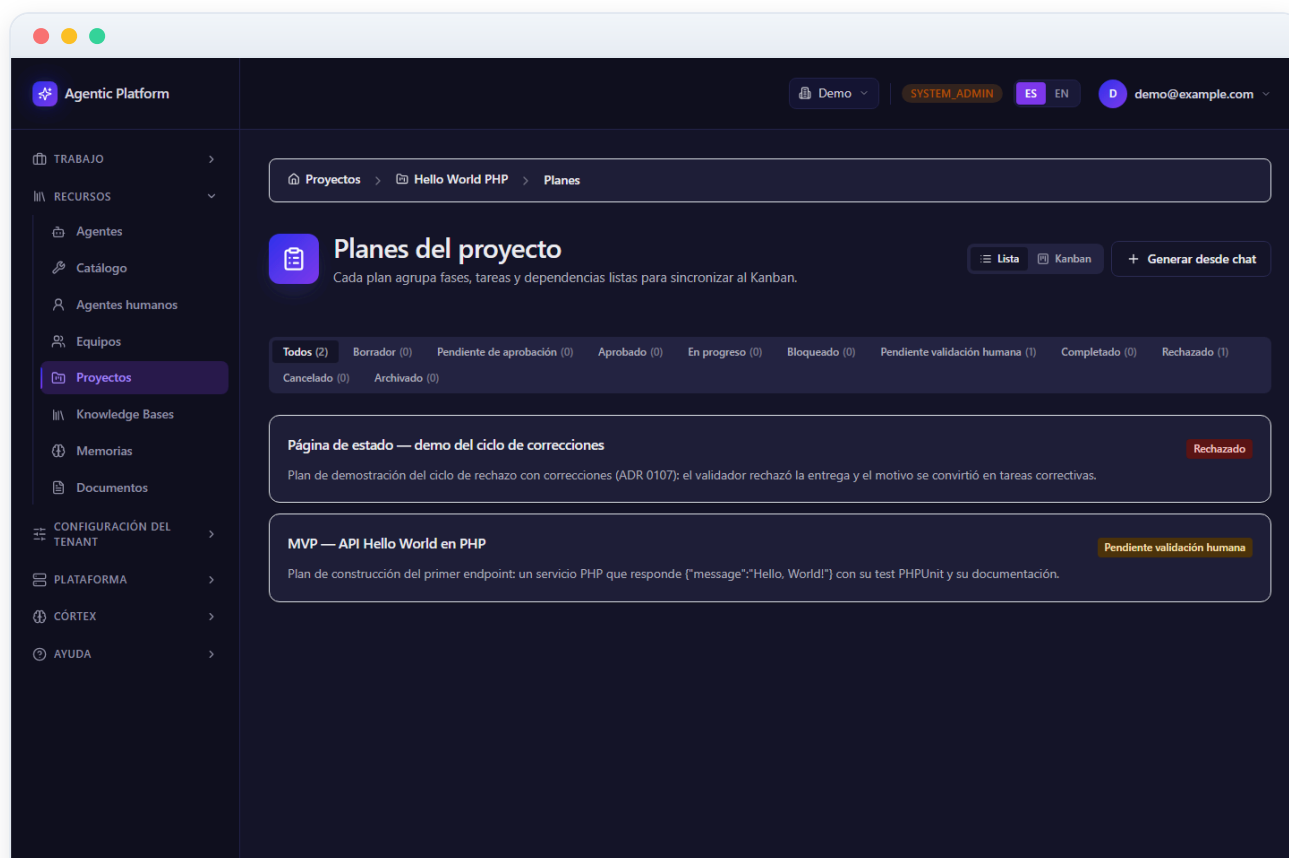


Figura 01.8 — Planes del proyecto

9 Tareas del proyecto

La sección **Tasks** reúne **todas** las tareas del proyecto, pertenezcan o no a un plan — incluye por tanto las tareas sueltas que se crean fuera del flujo de planning. Para el plan de ejemplo verás cuatro tareas: definir el endpoint, implementar el controlador PHP, escribir el test PHPUnit y documentarlo.

- Cada tarea muestra su **título**, **estado** (backlog, ready, en curso, revisión, bloqueada, hecha...), **prioridad** (baja, media, alta, crítica) y, si está asignada, el **agente responsable**.
- Las tareas de un plan llevan además sus **criterios de aceptación**: condiciones concretas y verificables que el trabajo debe cumplir para darse por bueno (p. ej. «Devuelve 200», «Content-Type application/json»).

Cuándo usar esta vista: para auditar el trabajo del proyecto en conjunto. Para la operativa diaria de un plan concreto es más cómodo el **Tablero** (doble Kanban), que se explica en el manual 02.

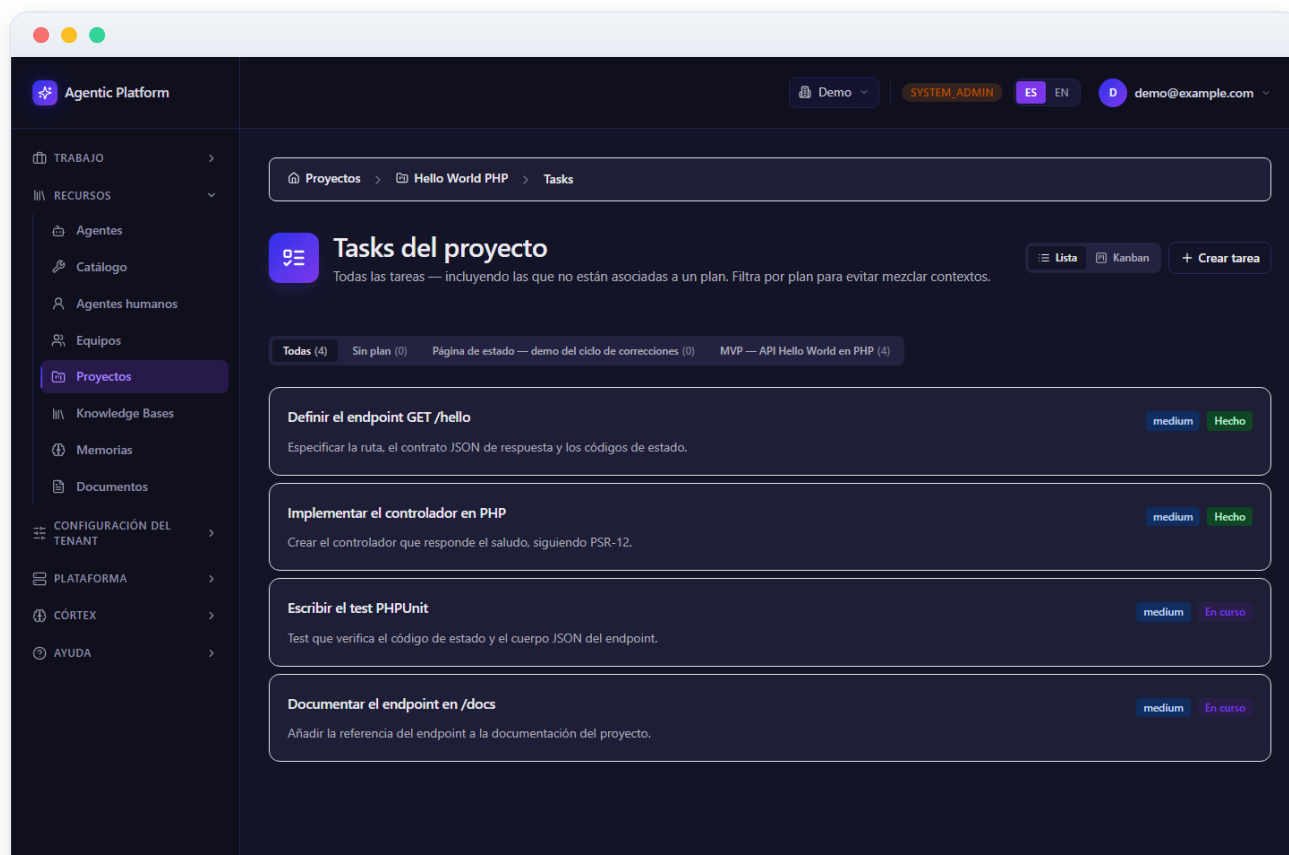


Figura 01.9 — Tareas del proyecto

10 Bases de conocimiento del proyecto (RAG)

Aquí se gestionan las **bases de conocimiento** (RAG) concedidas al proyecto: la documentación, convenciones, especificaciones y material de referencia que los agentes consultan mientras trabajan. Un agente que «sabe» las convenciones del proyecto produce código alineado a la primera; por eso esta sección importa tanto.

- Las bases se **conceden** desde el catálogo del tenant o desde las built-in de la plataforma: el proyecto no duplica contenido, sino que recibe acceso.
- Los documentos se **indexan automáticamente**: se trocean y se generan embeddings (vectores en pgvector) para que la búsqueda semántica encuentre pasajes relevantes aunque no coincidan las palabras exactas.
- Si creaste el proyecto desde una **plantilla**, las bases de la plantilla pueden haberse concedido automáticamente (la casilla del asistente de creación).

Buena práctica: concede solo las bases pertinentes al proyecto. Más conocimiento no siempre es mejor — el contexto irrelevante diluye las respuestas.

The screenshot displays the 'Agentic Platform' interface. On the left is a sidebar with a navigation menu. The main area is titled 'Knowledge Bases del proyecto' and contains instructions on how to add knowledge. Below this is a 'Catálogo de conocimiento' section with a list of knowledge bases, each with an 'Activar' button. The knowledge bases listed are:

- API REST design guidelines
- CodeIgniter 4 — Arquitectura HMVC y routing
- CodeIgniter 4 — CI/CD y despliegue
- CodeIgniter 4 — Convenciones del equipo
- CodeIgniter 4 — Estrategia de testing
- CodeIgniter 4 — Frontend y assets
- CodeIgniter 4 — Internacionalización EN/ES
- CodeIgniter 4 — Modelo de datos con Doctrine
- CodeIgniter 4 — Seguridad y autenticación
- Node + Express conventions
- PHP + Symfony conventions
- PostgreSQL best practices
- Python + FastAPI conventions
- React + Next.js conventions

Figura 01.10 — Bases de conocimiento del proyecto (RAG)

11 Comandos autorizados y runtime por defecto

La sección **Comandos & runtime** gobierna qué puede ejecutar el equipo de agentes en el stack del proyecto. Tiene dos partes:

- **Comandos autorizados:** la **lista blanca** de binarios que los agentes pueden lanzar vía `shell_exec`. Es *deny-by-default*: con la lista vacía no se ejecuta nada. Los comandos se muestran como **chips** que puedes quitar uno a uno; el campo de texto añade nuevos, y los botones de **preset por stack** (PHP, Node, .NET, Python) rellenan de un golpe los binarios típicos de cada stack (p. ej. PHP añade `php`, `composer`, `vendor/bin/phpunit` y `pest`) sin borrar los que ya tuvieras.
- **Runtime por defecto:** el *runtime template* del proyecto — aquí `php-phpunit` —, elegido de un catálogo mantenido por la plataforma. Los runtimes son **contenedores efímeros y aislados** (sin red abierta, sin privilegios) donde se ejecutan los tests y comandos del stack; los workers de la plataforma solo orquestan, nunca ejecutan código del usuario.

La edición requiere rol de **administrador de tenant**; el resto de miembros ve la configuración en modo lectura.

Buena práctica: autoriza el mínimo imprescindible y amplía bajo demanda; cada binario añadido es superficie de ejecución extra.

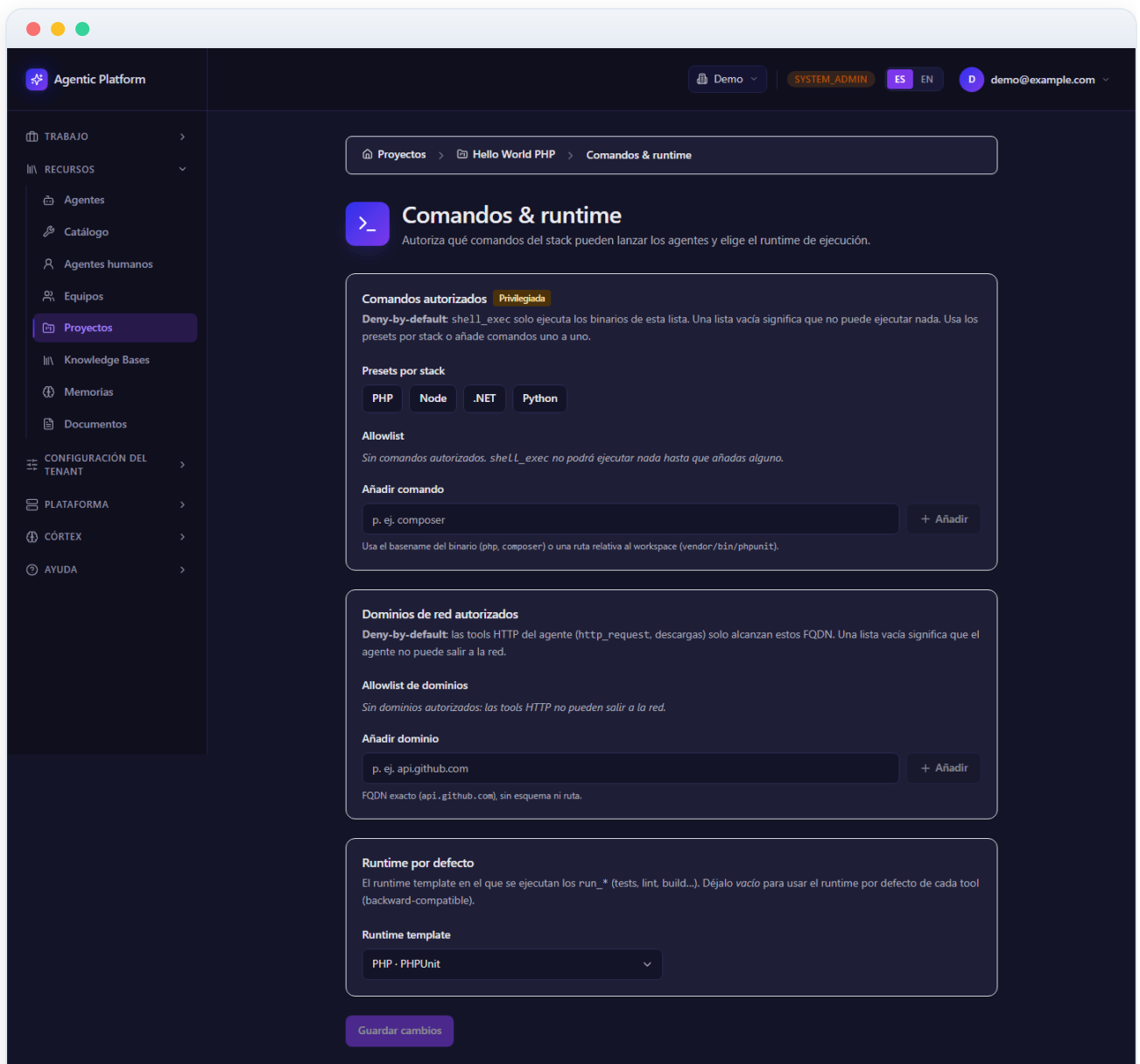


Figura 01.11 — Comandos autorizados y runtime por defecto

12 Caché de dependencias por runtime

La sección **Caché de dependencias** muestra, por cada runtime template usado por el proyecto, la caché de dependencias descargadas (paquetes de composer, npm, pip...) que la plataforma conserva entre ejecuciones para no re-descargarlo todo en cada tarea.

- La tabla lista los **runtimes con caché** y permite **invalidarla** por runtime: la próxima ejecución partirá de cero y volverá a resolver las dependencias.
- Invalidar es útil cuando una dependencia corrupta o desactualizada provoca fallos raros en los tests: es el equivalente a «borrar node_modules y reinstalar».

En un proyecto recién creado esta lista estará vacía: la caché se va poblando a medida que los agentes ejecutan instalaciones de dependencias en los runtimes.

The screenshot shows the 'Agentic Platform' interface. The top navigation bar includes 'Demo', 'SYSTEM_ADMIN', 'ES', 'EN', and a user profile 'demo@example.com'. The left sidebar has a menu with 'TRABAJO', 'RECURSOS', 'Agentes', 'Catálogo', 'Agentes humanos', 'Equipos', 'Proyectos' (selected), 'Knowledge Bases', 'Memorias', 'Documentos', 'CONFIGURACIÓN DEL TENANT', 'PLATAFORMA', 'CÓRTEX', and 'AYUDA'. The main content area is titled 'Caché de dependencias' and includes a breadcrumb 'Proyectos > Hello World PHP > Caché de dependencias'. Below the title is a description: 'Invalida la caché del dep-cache para forzar al worker-test a reinstalar las dependencias en el siguiente run. Útil cuando sospechas que la caché está corrupta.' A table titled 'Runtimes con caché' lists various runtimes and their cache paths, with an 'Invalidar' button for each.

RUNTIME	PUNTO DE MONTAJE	ACCIONES	RESULTADO
Python · pytest	/home/agent/.cache/pip	Invalidar	
Node · Jest	/home/agent/.npm	Invalidar	
Node · Vitest	/home/agent/.npm	Invalidar	
Node · Playwright	/home/agent/.npm	Invalidar	
PHP · PHPUnit	/home/agent/.composer/cache	Invalidar	
PHP · Pest	/home/agent/.composer/cache	Invalidar	
Go · go test	/home/agent/go/pkg/mod	Invalidar	
Java · Maven	/home/agent/.m2/repository	Invalidar	
Java · Gradle	/home/agent/.gradle/caches	Invalidar	
Ruby · RSpec	/home/agent/.bundle	Invalidar	
Rust · Cargo	/home/agent/.cargo/registry	Invalidar	
.NET · dotnet test	/home/agent/.nuget/packages	Invalidar	

Figura 01.12 — Caché de dependencias por runtime

13 Servidores MCP del proyecto

La sección **MCP servers** conecta el proyecto con servidores **Model Context Protocol**: integraciones externas (repositorios, bases de datos, servicios de terceros, buscadores...) que exponen herramientas invocables por los agentes de forma gobernada.

- Cada servidor MCP **declara sus herramientas**; una vez registrado, esas herramientas entran en el catálogo y se asignan **por agente** — un agente solo puede usar las herramientas que tiene concedidas.
- Las credenciales de los servidores no se guardan en claro: la plataforma usa su gestor de secretos (Vault) como única vía de credenciales.
- La vista **Tools por agente** (más abajo) permite auditar el resultado final de estas asignaciones.

Caso de uso: conectar el MCP de tu gestor de incidencias para que el equipo de agentes pueda leer tickets y enlazarlos con las tareas del plan.

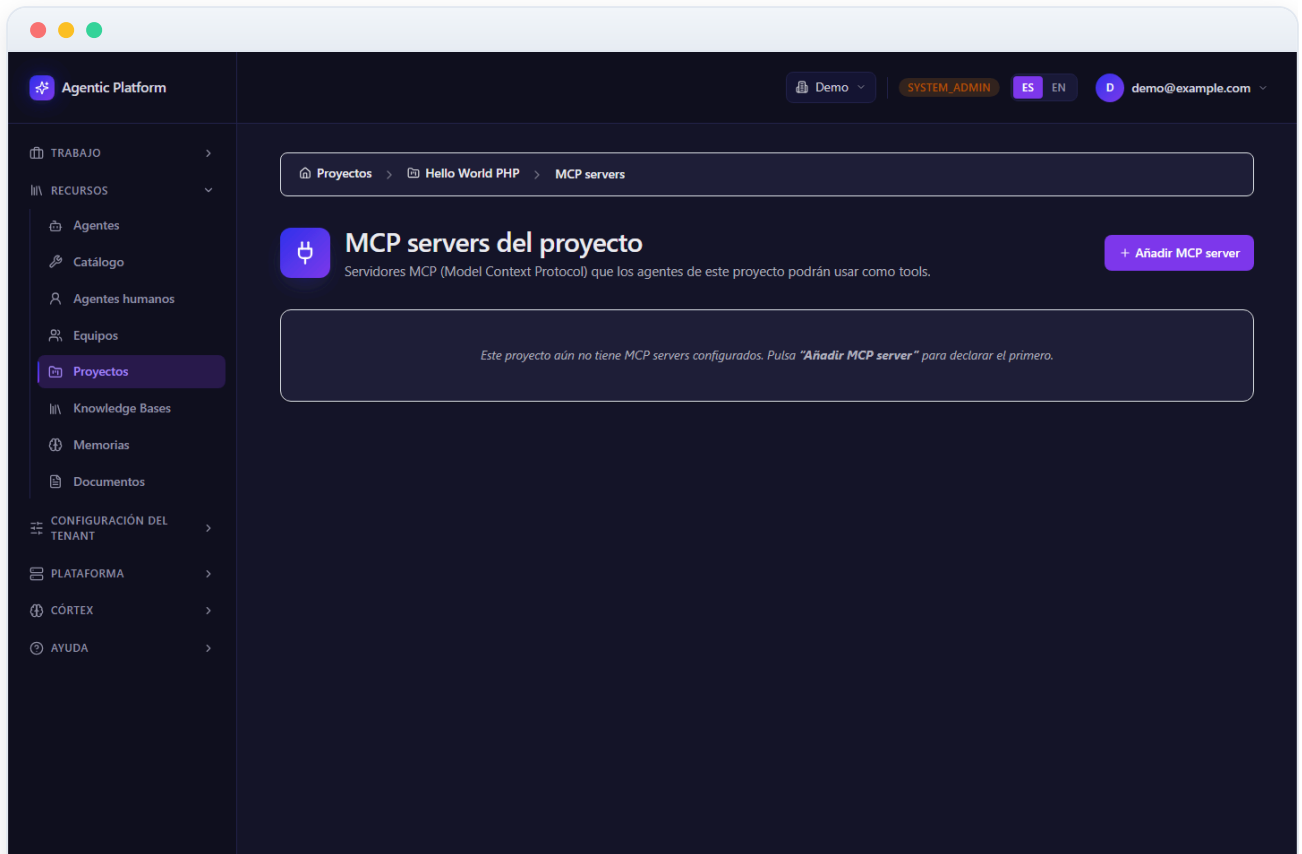


Figura 01.13 — Servidores MCP del proyecto

14 Webhooks entrantes del proyecto

La sección **Webhooks entrantes** permite que herramientas externas (GitHub, Jira, Sentry...) **disparen acciones** en el proyecto enviando eventos a una URL dedicada. Es la vía para reaccionar automáticamente a un push, a un ticket nuevo o a una alerta de errores.

- Cada webhook se configura con un **origen**, un **nombre** y una o varias **reglas evento** → **acción** (qué evento externo dispara qué acción en la plataforma).
- Al crearlo se genera un **secreto HMAC** que se muestra *una sola vez* (banner con botón de copiar): configúralo en la herramienta externa. Toda entrega entrante se **verifica por firma** antes de actuar — las peticiones sin firma válida se descartan.
- Cada tarjeta de webhook muestra su URL completa, su estado (activado/desactivado) y acciones de **editar**, **rotar el secreto** y **borrar**; un desplegable lista las **entregas recientes** para depurar integraciones.

Aviso: si rotas el secreto, actualízalo también en la herramienta externa o sus entregas empezarán a rechazarse.

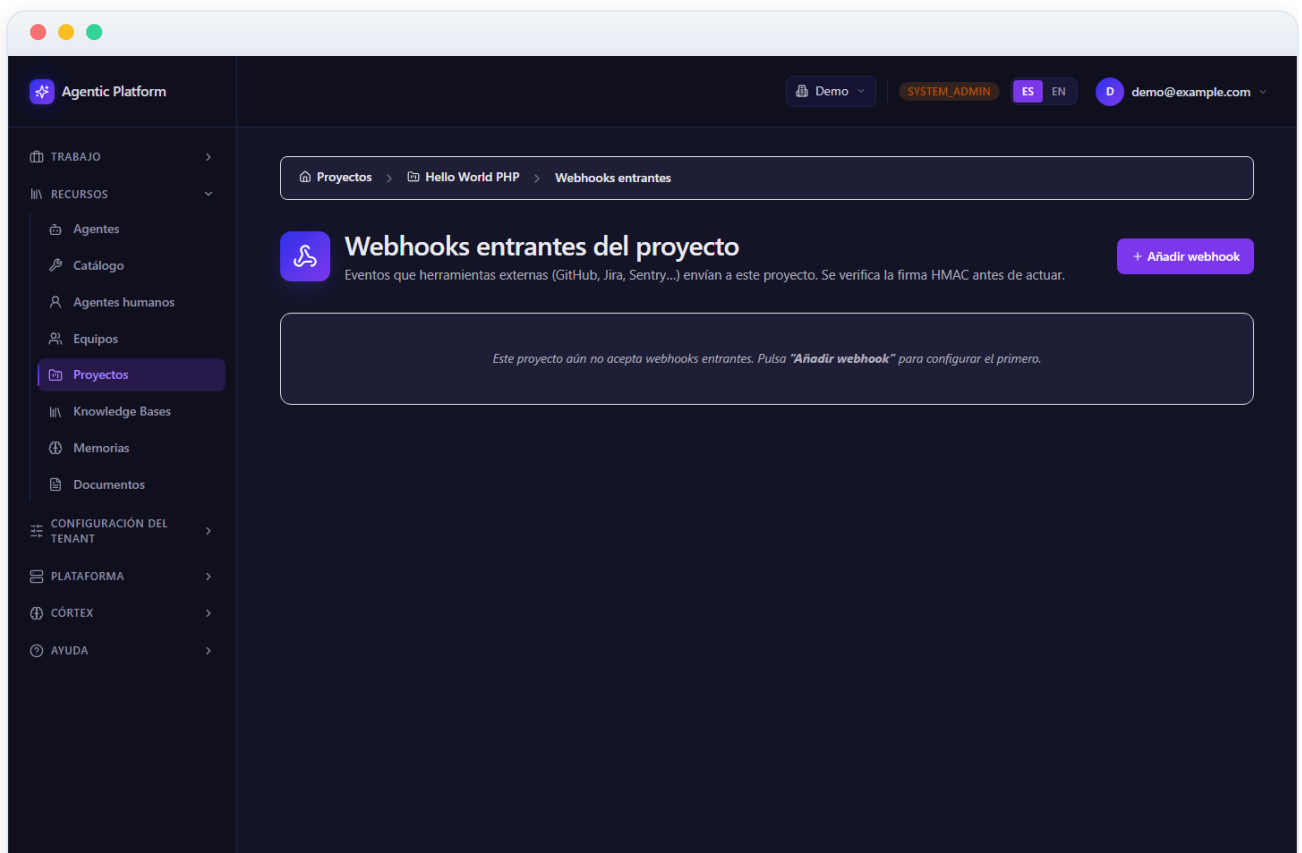


Figura 01.14 — Webhooks entrantes del proyecto

15 Memoria del proyecto

La **Memoria** del proyecto es lo que el equipo «recuerda» en el ámbito **project_shared**: decisiones tomadas, hechos aprendidos, convenciones acordadas y lecciones de ejecuciones anteriores, que persisten entre ejecuciones y alimentan el contexto de los agentes.

- La plataforma distingue **cuatro ámbitos de memoria** y nunca los mezcla: *private* (personal del usuario humano — un agente de IA ni la lee ni la escribe), *team_shared* (del equipo), *project_shared* (de este proyecto — lo que ves aquí) y *global* (de la organización).
- Las memorias se crean tanto automáticamente (los agentes registran aprendizajes relevantes) como manualmente desde esta pantalla.
- Puedes revisar, editar o eliminar entradas: la memoria es gobernable, no una caja negra.

Buena práctica: purga periódicamente las memorias obsoletas (decisiones revertidas, datos caducos). Una memoria limpia produce agentes más precisos.

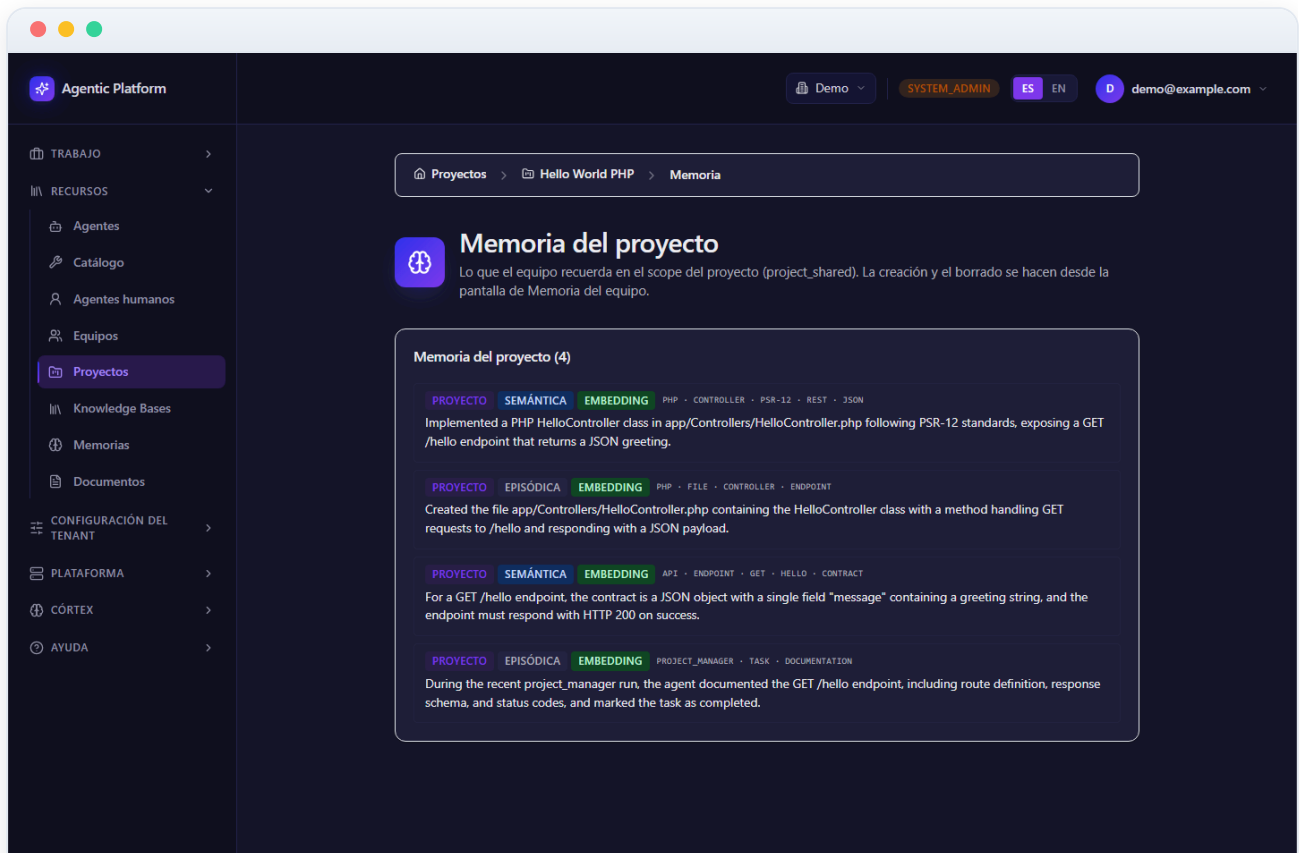


Figura 01.15 — Memoria del proyecto

16 Diagnóstico de herramientas por agente

Esta vista de **solo lectura** muestra, para cada agente del proyecto, qué **herramientas** tiene efectivamente asignadas y con qué variante (p. ej. sandboxed). Es el resultado final de combinar el catálogo del tenant, los servidores MCP del proyecto y las asignaciones por agente.

- **Para qué sirve:** auditar de un vistazo qué puede hacer cada agente *antes* de lanzar una ejecución — si el QA no tiene la herramienta de ejecutar tests, mejor descubrirlo aquí que a mitad de un plan.
- Al ser diagnóstico, **no se edita nada** desde esta pantalla: las asignaciones se cambian en el catálogo de herramientas y en la ficha de cada agente.

Caso de uso: un plan se bloquea porque el agente no puede escribir ficheros → entra aquí, comprueba qué variante de herramienta de escritura tiene concedida y corrige la asignación.

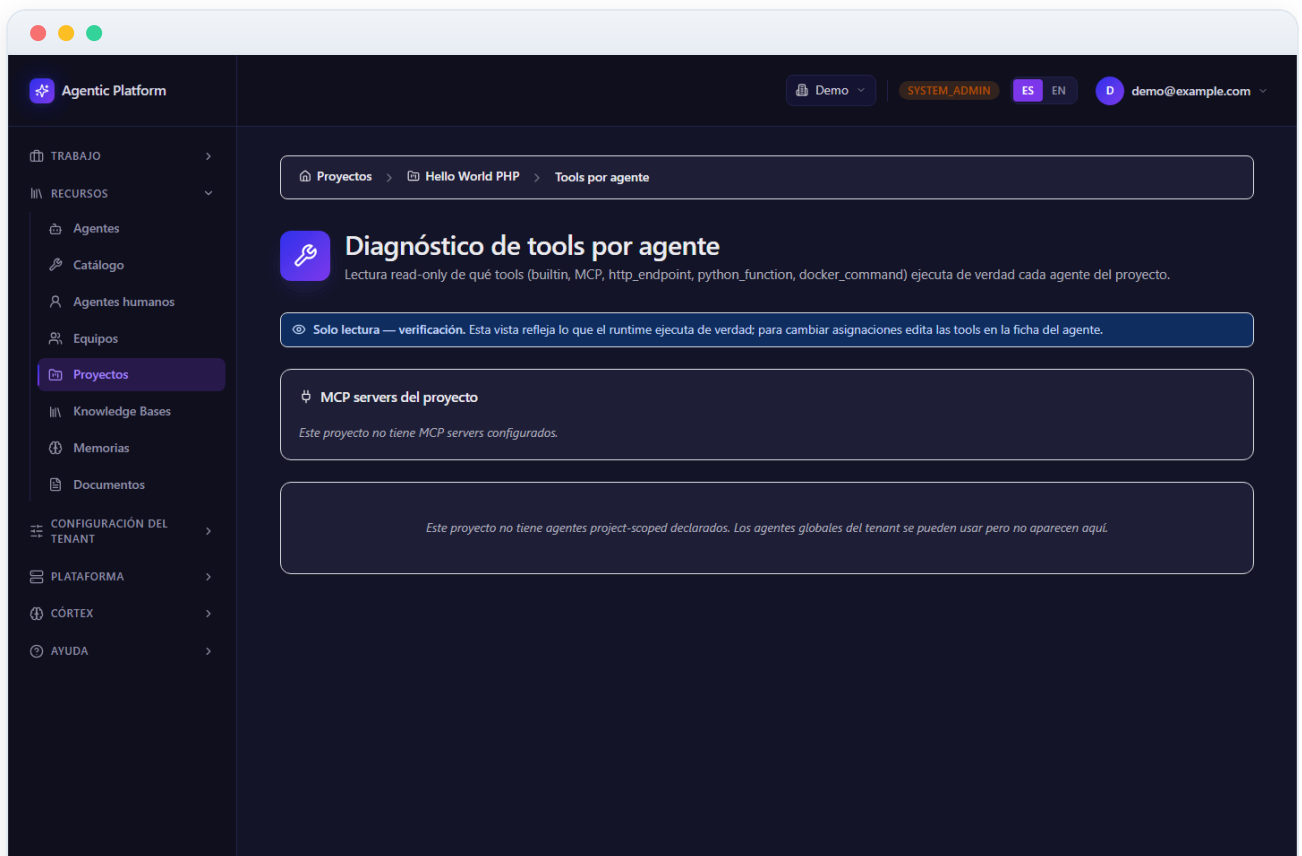


Figura 01.16 — Diagnóstico de herramientas por agente